

PLACEMENT PORTAL APPLICATION REPORT

1.Student Details

Name: Mrigaank Sharma

Roll Number: 23F2003457

Email: 23f2003457@ds.study.iitm.ac.in

About Me: I am a Diploma level student in the IIT Madras BS Degree Program with a keen interest in Web Development, Artificial Intelligence, Machine Learning, and Generative AI. I enjoy developing practical applications that integrate modern technologies to solve real-world problems while continuously improving my software development skills.

2.Project Details

Project Title: **PLACEMENT PORTAL APPLICATION**

Problem Statement: To design and build a web-based Placement Portal System that enables administrators, companies, and students to **manage** placement activities efficiently. The application allows students to **search** and **apply** for placement drives, while administrators and companies manage drives, applications, interviews, and selections. It also prevents duplicate applications, **tracks** application status, and maintains complete placement history.

Approach: The application was developed using Flask with a modular API architecture and SQLite for database management. It provides separate dashboards for Admin, Company, and Students, enabling company management, drive approvals, application management, interview scheduling, and secure role-based authentication through a responsive web interface.

3. AI/LLM Declaration

I used **ChatGPT** to assist in writing SQLAlchemy model definitions, creating API documentation samples, and improving variable naming consistency.

The extent of AI/LLM usage is around **15–18%**, limited to **code suggestions and documentation formatting**.

All final implementation logic, debugging, and integration were done manually.

4. Technologies and Frameworks Used

Technology /Library Purpose

Flask	Backend API framework
SQLAlchemy	Database ORM
SQLite	Local database
VueJS	Frontend UI framework
Bootstrap 5	Responsive UI design
Axios	API communication
JWT	User authentication and role-based access

5.Database Schema / ER Diagram

Tables:

1. **User** — Stores common user information (*id, name, email, password, role, is_active, created_at*).
2. **StudentProfile** — Stores student-specific information (*id, user_id, student_id, phone, branch, cgpa, graduation_year, resume, is_placed*).
3. **CompanyProfile** — Stores company information (*id, user_id, company_name, hr_name, hr_email, phone, website, description, approval_status, is_blacklisted*).
4. **PlacementDrive** — Stores placement drive details (*id, company_id, job_title, job_description, eligibility_branch, minimum_cgpa, graduation_year, application_deadline, salary, location, employment_type, vacancies, status, created_at*).
5. **Application** — Stores student placement applications (*id, student_id, drive_id, applied_at, status*).
6. **Interview** — Stores interview details for applications (*id, application_id, interview_date, interview_type, status, remarks*).

7. **Notification** — Stores user notifications (*id, user_id, message, is_read, created_at*).

Relationships:

- **One-to-One** → **User** → **StudentProfile** (One student profile belongs to one user.)
- **One-to-One** → **User** → **CompanyProfile** (One company profile belongs to one user.)
- **One-to-Many** → **CompanyProfile** → **PlacementDrive** (One company can create multiple placement drives.)
- **One-to-Many** → **StudentProfile** → **Application** (One student can submit multiple applications.)
- **One-to-Many** → **PlacementDrive** → **Application** (One placement drive can receive multiple applications.)
- **One-to-One/One-to-Many** → **Application** → **Interview** (An application can have interview records.)
- **One-to-Many** → **User** → **Notification** (One user can receive multiple notifications.)
- **Unique Relationship** → **StudentProfile + PlacementDrive** → **Application** (A student cannot apply to the same placement drive more than once.)



6. API Resource Endpoints

Endpoint	Method	Description
/api/auth/login	POST	Authenticate user and generate JWT token
/api/auth/register/student	POST	Register a new student
/api/admin/drives/<drive_id>/approve	PUT	Approve a placement drive

Endpoint	Method	Description
/api/student/drives	GET	View approved placement drives

YAML API Definition File:

Included separately in the submission ZIP as api.yaml.

7. Architecture and Features

Architecture Overview

- **run.py** – Main Flask application entry point.
- **app/** – Main Flask application package containing the backend modules.
- **app/models.py** – SQLAlchemy database models for users, students, companies, drives, applications, interviews, and notifications.
- **app/auth, app/admin, app/company, app/student** – Flask modules for authentication and role-based functionality.
- **frontend/src/** – VueJS frontend containing pages, components, layouts, services, and routing.
- **app/tasks/** – Celery background tasks for reminders, reports, and CSV exports.
- **celery_worker.py** – Celery worker configuration and Flask application context.
- **Redis** – Used for caching and Celery message brokering.
- **SQLite** – Application database.

Implemented Features

- **User registration and secure login.**
- **Role-based access control (Admin, Company, Student).**
- **Company registration and admin approval/rejection.**
- **Placement drive creation and admin approval/rejection.**
- **Student eligibility validation and placement-drive application.**
- **Application management and status updates.**
- **Interview scheduling and final selection management.**

- **Student profile and resume management.**
- **Search and filtering of placement drives and users.**
- **Prevention of duplicate applications.**
- **Placement history and notifications.**
- **Responsive user interface using VueJS and Bootstrap 5.**

Additional Features

- **Redis caching with cache expiry.**
- **Celery-based asynchronous CSV application export.**
- **Daily deadline reminder notifications and email alerts.**
- **Automated monthly placement activity reports sent to Admin via email.**
- **Celery Beat for scheduled background jobs.**
- **Admin placement statistics and management dashboard.**
- **Interview and selection tracking.**
- **Secure password hashing and JWT-based authentication.**

8. Video Presentation

Drive Link: <https://drive.google.com/file/d/1b-fl9lc5XfxotDpDYeq5EdxLop-6ETM3/view?usp=sharing>

Github link: <https://github.com/MRIGAANKSh/iitm-mad2-project.git>